

Analiza pokreta i praćenje objekata



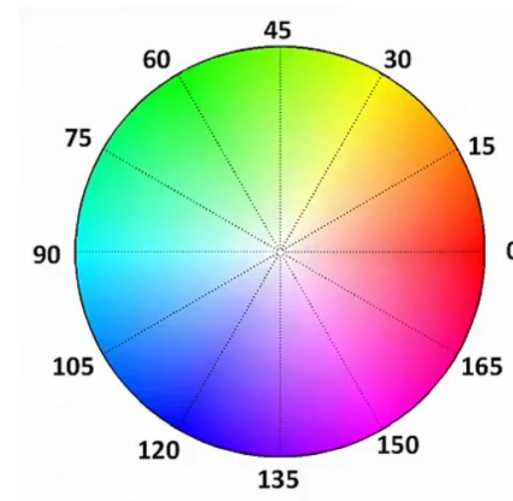
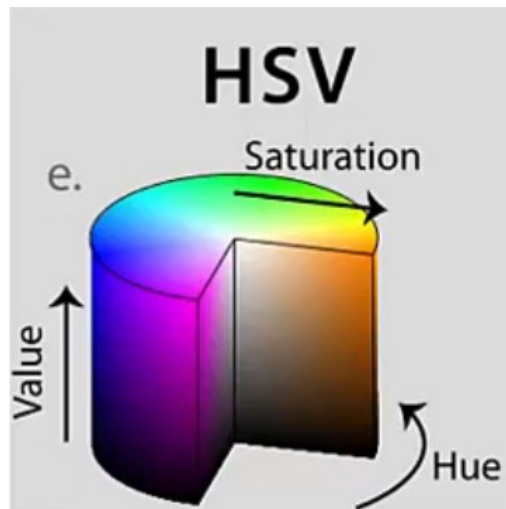
Sadržaj

- Ponovimo kroz primjere...
 - Filtiranje po boji (*filtering by color*)
 - Oduzimanje pozadine i detekcija prednjeg plana (*background subtraction, foreground detection*)
- Algoritmi za praćenje pokrenih objekata u sceni (*object tracking*)
 - Meanshift
 - Camshift
 - Optical flow

Filtriranje po boji

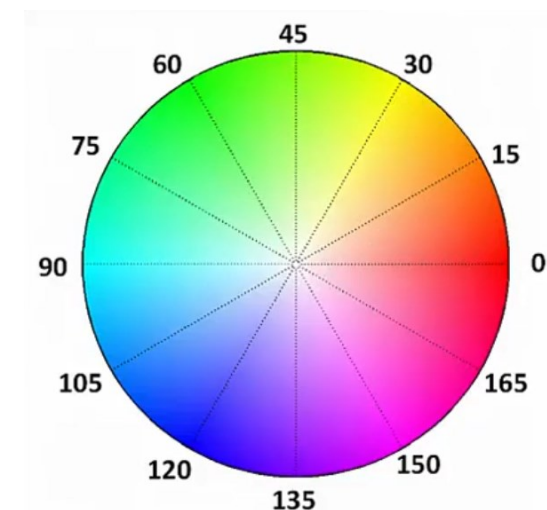
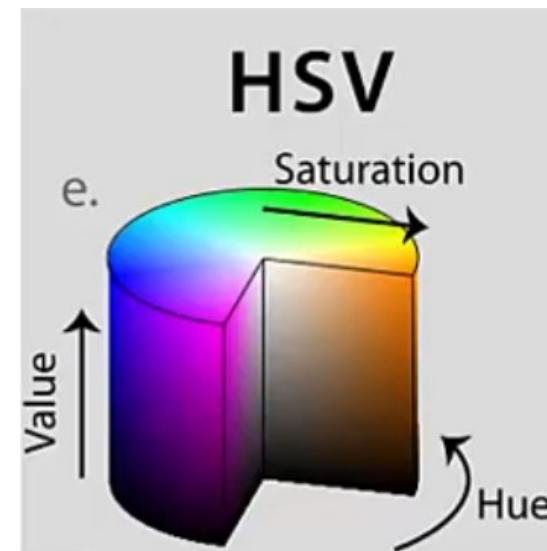
Filtriranje po boji

- Zašto je filtriranje po boji dio ovog predavanja (analiza i praćenje objekta) ?
 - Vrlo korisna tehnika za praćenje objekta
 - Ako znamo boju objekta kojeg želimo pratiti, lako ga možemo odvojiti od ostatka scene
- Načini zapisa boje – podsjetimo se... (RGB, YUV, HSV,...)
- HSV omogućava jednostavno filtriranje objekata prema boji



Filtriranje po boji & HSV sustav boja

- Filtriranje prema *Hue* komponenti (boja)
 - Hue → opseg boja ide od 0 do 180
 - Filtri za pojedinu boju:
 - Crvena – 165 do 15
 - Zelena – 45 do 75
 - Plava – 90 do 120
- Filtriranje prema *Saturation* i *Value/Brightness*
 - Obično se postavlja opseg filtra od 50 do 255 za zasićenje i osvjetljenje jer time zadržavamo zapravo stvarne boje
 - *Saturation* od 0 do 50 je vrlo blizu bijele boje (svjetlije boje)
 - *Value* od 0 do 50 je vrlo blizu crne boje (tamnije boje)



Filtriranje po boji u Open CV

- Primjer 1 (Python & OpenCV)
 - Detekcija i praćenje objekta poznate boje koristeći web kameru
- Obratite pažnju na količinu samog koda
- Standardne *import* naredbe i naredbe za otvaranje/zatvaranje prozora
- Inicijalizacija web kamere
- Definicija opsega filtra u HSV prostoru
- Pretvorba iz RGB/BGR u HSV
- *frame* – slika prije filtriranja
- *mask* – binarna slika koja ima „1“ samo gdje su boje u definiranom opsegu
- *res* – ulazna slika samo s bojom u definiranom opsegu

```
import cv2
import numpy as np

# Initialize webcam
cap = cv2.VideoCapture(0)

# define range of YELLOW color in HSV
lower_yellow = np.array([25,50,50])
upper_yellow = np.array([35,255,255])

# loop until break statement is executed
while True:

    # Read webcam image
    ret, frame = cap.read()

    # Convert image from RGB/BGR to HSV so we easily filter
    hsv_img = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # Use inRange to capture only the values between lower & upper_yellow
    mask = cv2.inRange(hsv_img, lower_yellow, upper_yellow)

    # Perform Bitwise AND on mask and our original frame
    res = cv2.bitwise_and(frame, frame, mask=mask)

    cv2.imshow('Original', frame)
    cv2.imshow('mask', mask)
    cv2.imshow('Filtered Color Only', res)
    if cv2.waitKey(1) == 13: #13 is the Enter Key
        break

cap.release()
cv2.destroyAllWindows()
```

P i t a n j a ?

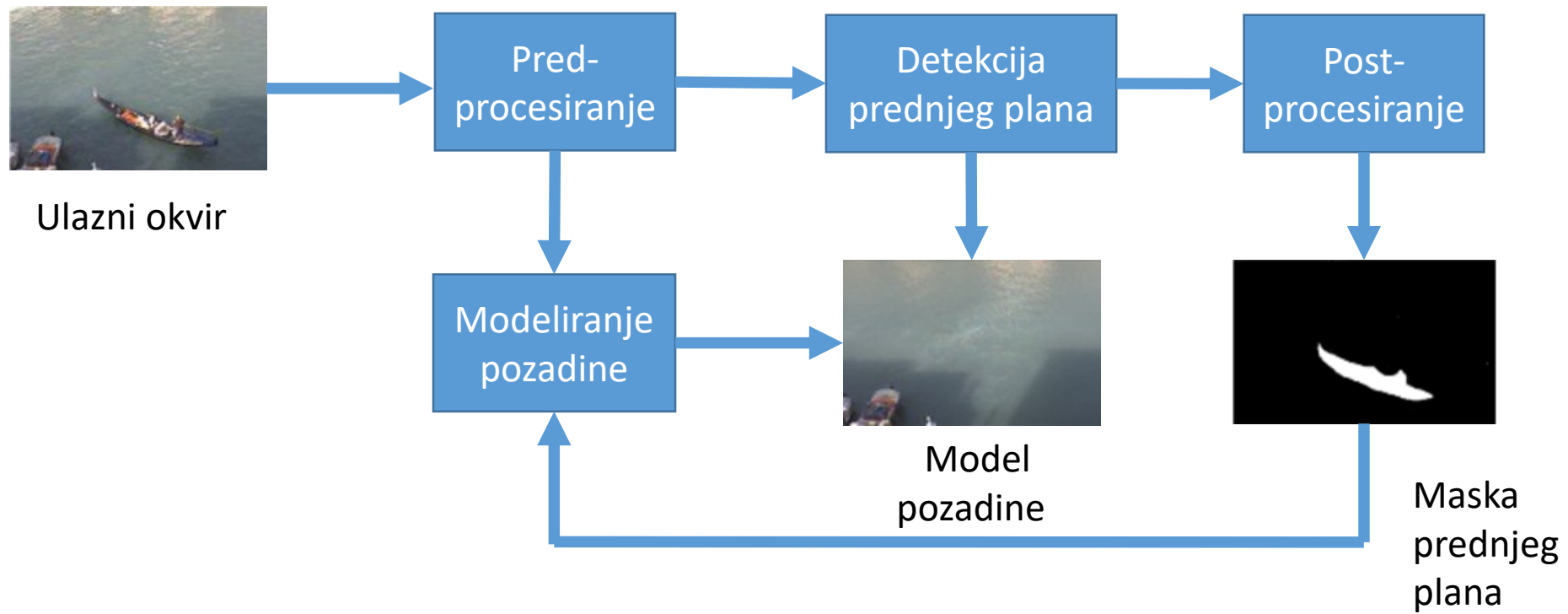
**Oduzimanje pozadine i
izdvajanje prednjeg plana**

Oduzimanje pozadine i izdvajanje prednjeg plana

- *Background subtraction* - BS
- Često se naziva i detekcija prednjeg plana (engl. *foreground detection*)
 - Od ulazne slike oduzima se pozadina kako bi se dobili pokretni objekti
- BS je jedan od glavnih koraka predobrade u mnogima aplikacijama zasnovanim na računalnom vidu
- **Vrlo korisna tehnika za detekciju pokretnih objekata u videu sa statičnom kamerom**
- Općenito, područja interesa u slici su objekti (ljudi, auti, tekst, ...)

Oduzimanje pozadine i izdvajanje prednjeg plana

Dijagram toka općenitog BS algoritma



Oduzimanje pozadine i izdvajanje prednjeg plana

Oduzimanje pozadine (Python & OpenCV)

- Implementirana 3 različita BS algoritma u OpenCV-u
 - *BackgroundSubtractorMOG*
 - *BackgroundSubtractorMOG2* (bolja prilagodba na promjene osvjetljenja i na detekciju sjene)
 - *Geometric Multigrid*



Oduzimanje pozadine i izdvajanje prednjeg plana

Primjer 2 (Python & OpenCV) - *MoG Background/Foreground segmentation algorithm*

```
# OpenCV 2.4.13 only
import numpy as np
import cv2

cap = cv2.VideoCapture('walking.avi')

# Initlaize background subtractor
foreground_background = cv2.BackgroundSubtractorMOG()

while True:

    ret, frame = cap.read()

    # Apply background subtractor to get our foreground mask
    foreground_mask = foreground_background.apply(frame)

    cv2.imshow('Output', foreground_mask)
    if cv2.waitKey(1) == 13:
        break

cap.release()
cv2.destroyAllWindows()
```

- pozadina uspješno uklonjena
- ostali pokretni objekti

Oduzimanje pozadine i izdvajanje prednjeg plana

Primjer 4 (Python & OpenCV) - *MoG2 Background/Foreground segmentation algorithm*

```
# OpenCV 2.4.13
import numpy as np
import cv2

cap = cv2.VideoCapture('walking.avi')

# Initlaize background subtractor
foreground_background = cv2.BackgroundSubtractorMOG2()

while True:
    ret, frame = cap.read()

    # Apply background subtractor to get our foreground mask
    foreground_mask = foreground_background.apply(frame)

    cv2.imshow('Output', foreground_mask)
    if cv2.waitKey(1) == 13:
        break

cap.release()
cv2.destroyAllWindows()
```

- Nije tako učinkovita po pitanju kretanja/nekretanja pozadine (vidimo lagane pomake u pozadini)
- Vidimo detekciju sjena koja je ovdje prilično uspješna

Oduzimanje pozadine i izdvajanje prednjeg plana

Primjer 5 (Python & OpenCV) - *Foreground subtraction*

- Oduzimanje prednjeg plana
- Želimo da ostane samo statična pozadina
- Npr. nadzor prometa
- Ukloniti
 - Pješake
 - Automobile
 - Ostale pokretne objekte
- **Pogodno za kodiranje**
 - **Izdvojena pozadina**

```
import cv2
import numpy as np

# Initialize webcam and store first frame
cap = cv2.VideoCapture(0)
ret, frame = cap.read()

# Create a float numpy array with frame values
average = np.float32(frame)

while True:
    # Get webcam frame
    ret, frame = cap.read()

    # 0.01 is the weight of image, play around to see how it changes
    cv2.accumulateWeighted(frame, average, 0.01)

    # Scales, calculates absolute values, and converts the result to 8-bit
    background = cv2.convertScaleAbs(average)

    cv2.imshow('Input', frame)
    cv2.imshow('Disapearing Background', background)

    if cv2.waitKey(1) == 13: #13 is the Enter Key
        break

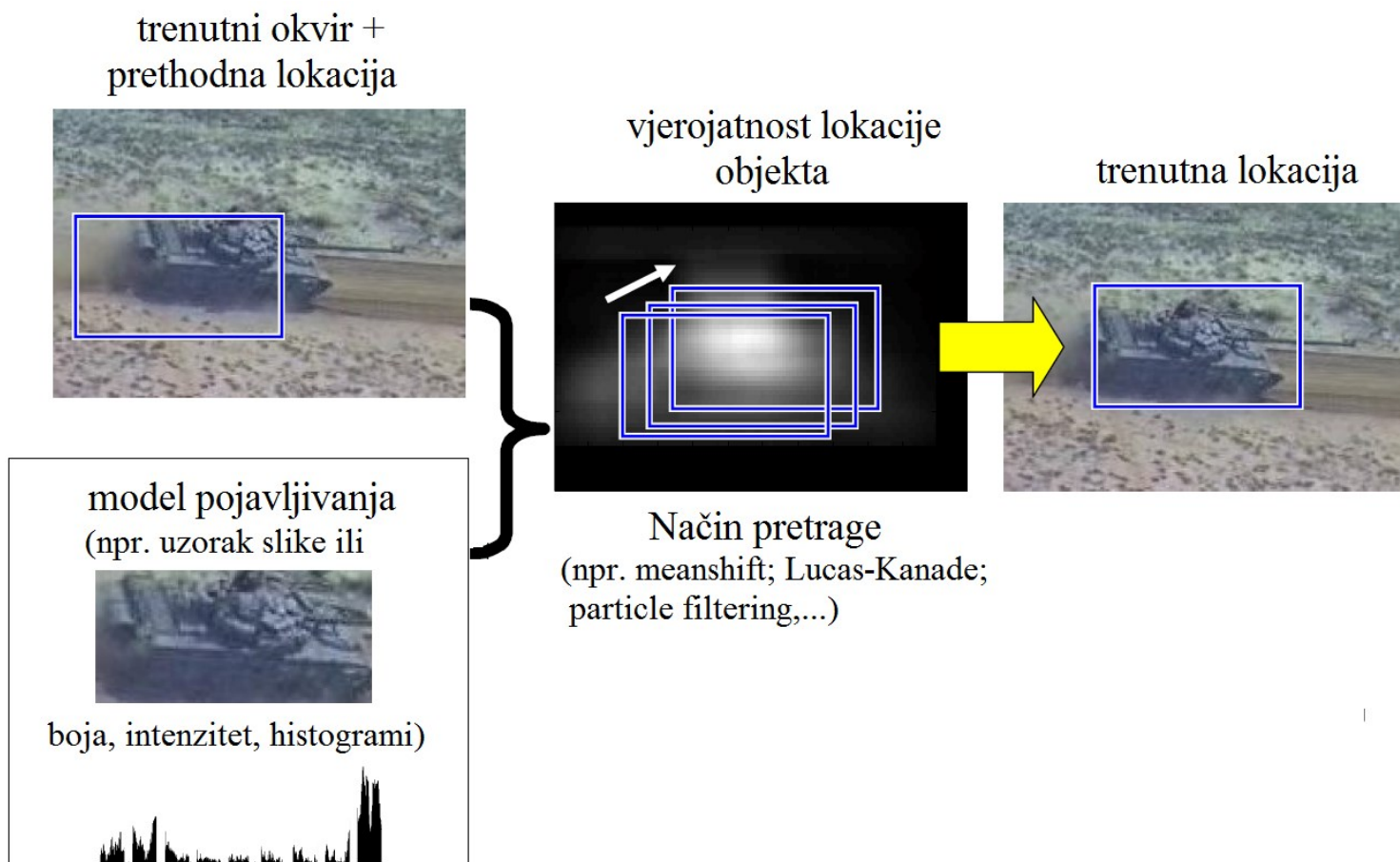
cv2.destroyAllWindows()
cap.release()
```

P i t a n j a ?

Algoritmi za praćenje pokrenih objekata u sceni

Algoritmi za praćenje pokrenih objekata u sceni

- Praćenje na temelju pojavljivanja (*appearance-based tracking*)



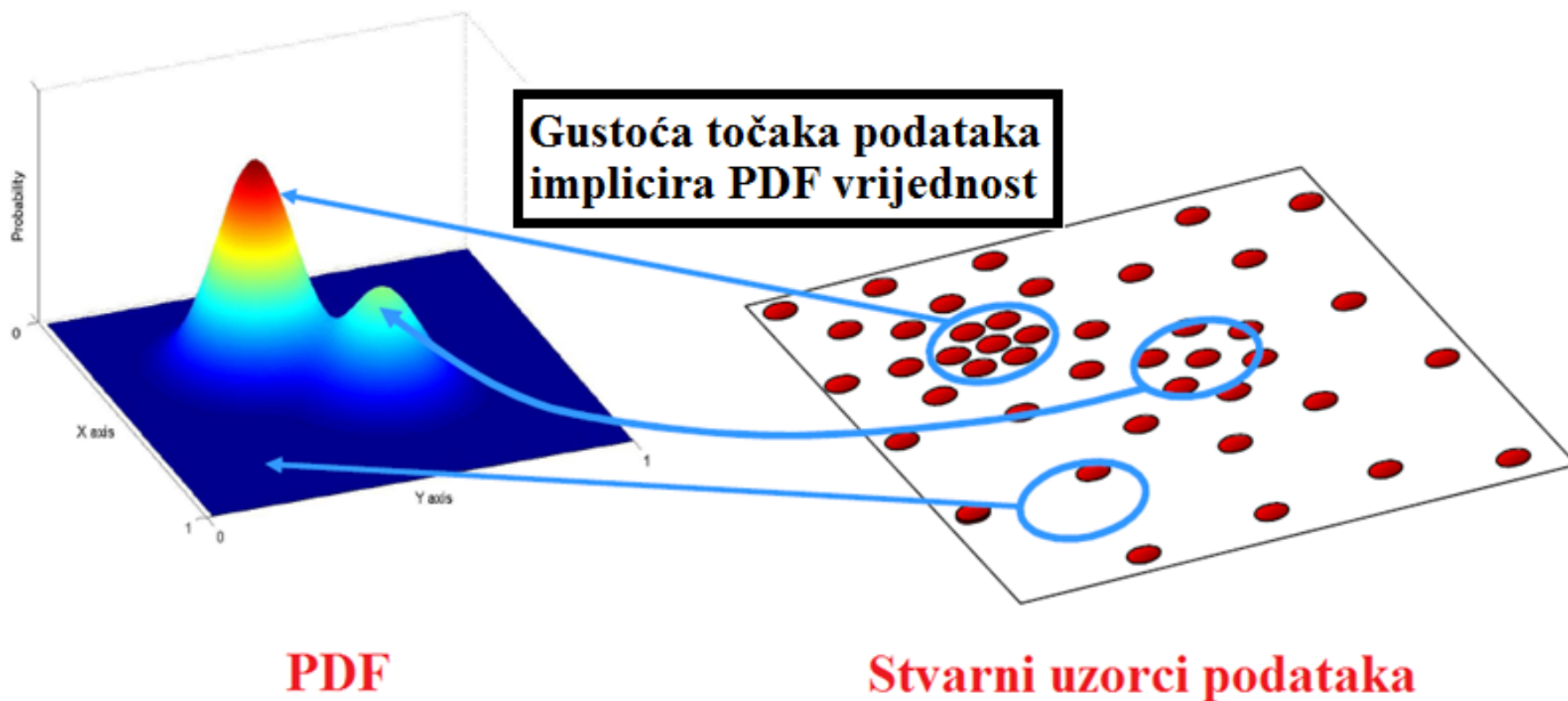
Meanshift algoritam

Meanshift algoritam za praćenje pokrenih objekata

- Motivacija – da bi pratili tzv. „ne-krute“ objekte (poput pješaka), teško je specificirati eksplicitni 2D parametarski model pokreta
- Pojavljivanje „ne-krutih“ objekata može ponekad biti modelirano raspodjelom boje
- Učinkovit pristup za praćenje objekata čija je pojava definirana određenom značajkom
 - Boja ili druge karakteristike, poput orijentacije rubova, teksture,...)

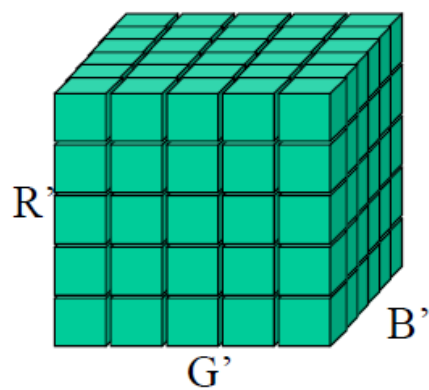
Meanshift algoritam za praćenje pokrenih objekata

- Prije *Maenshift-a* razmotrimo što je PDF (Probability Density Function) ?

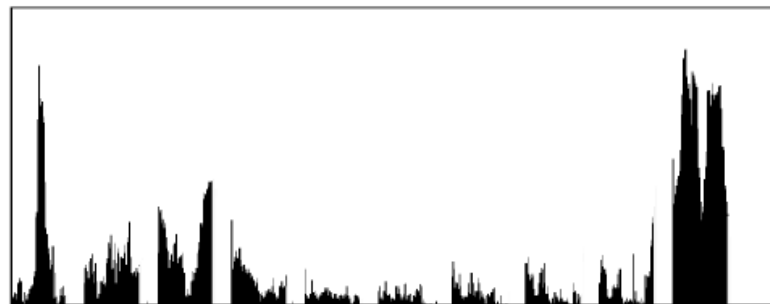


Meanshift algoritam za praćenje pokrenih objekata

- Npr. praćenje objekata koristeći histogram boja
 - Sjećamo li se što je histogram boja?



discretize



Raspodjela boje (1D normalizirani histogram)

$$R' = R \ll (8 - \text{nbits})$$

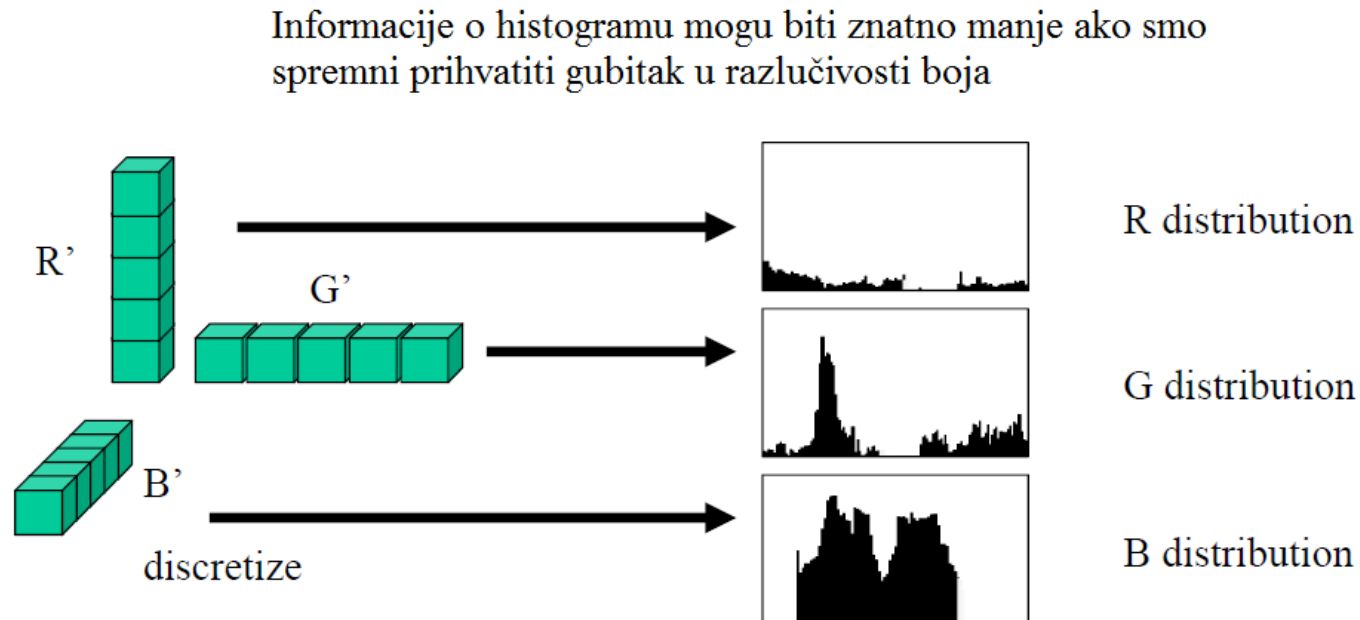
$$G' = G \ll (8 - \text{nbits})$$

$$B' = B \ll (8 - \text{nbits})$$

Ukupna veličina histograma $(2^{(8-\text{nbits})})^3$

Meanshift algoritam za praćenje pokrenih objekata

- Praćenje objekata koristeći histogram boja – manji histogrami boja



$$R' = R \ll (8 - \text{nbits})$$

$$G' = G \ll (8 - \text{nbits})$$

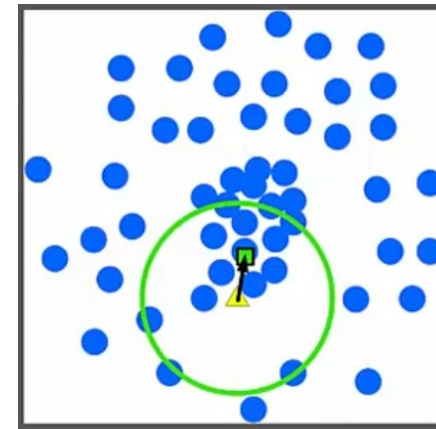
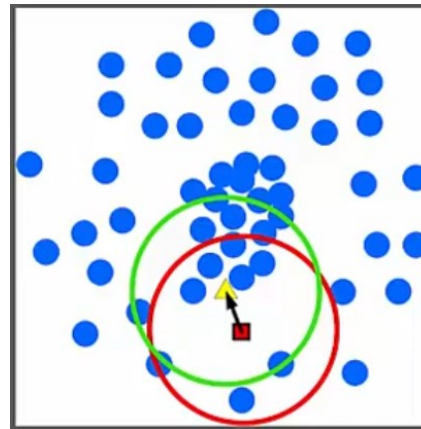
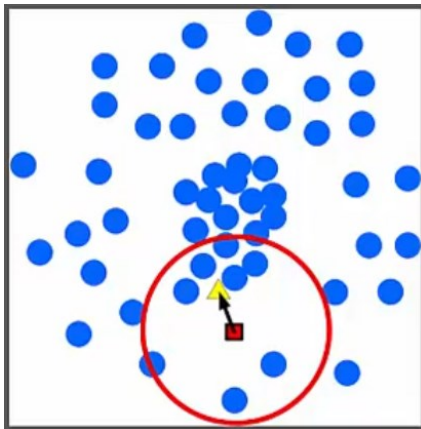
$$B' = B \ll (8 - \text{nbits})$$

Ukupna veličina histograma $3 * (2^{(8 - \text{nbits})})$

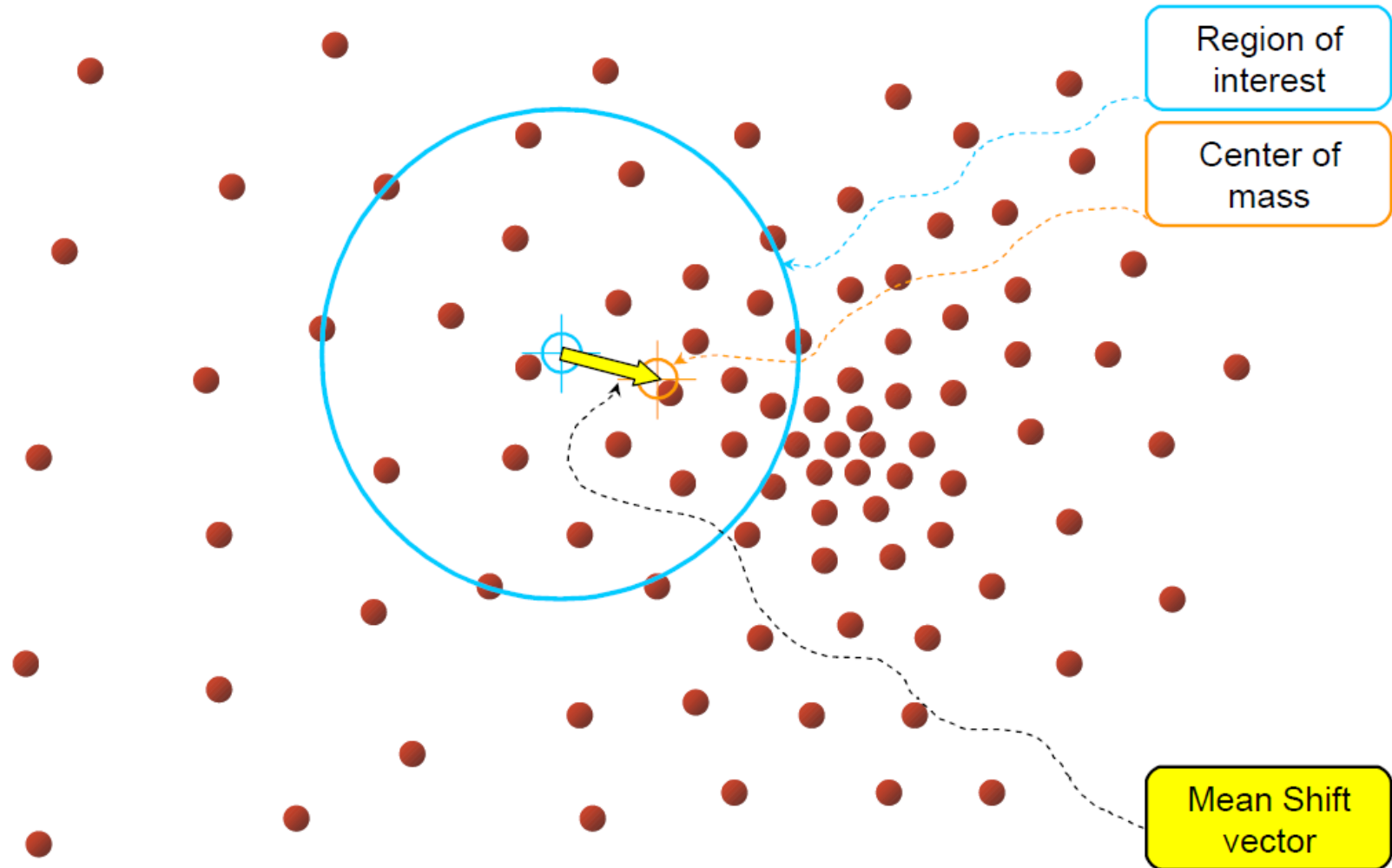
Meanshift algoritam za praćenje pokrenih objekata

Meanshift → njegova je pretpostavka jednostavna

- Neka imamo na raspolaganju skup točaka koje bi prema nekom kriteriju mogle pripadati objektu kojeg želimo pratiti
- Prati objekte pronalaženjem **maksimalne gustoće uzorka diskretnih točaka** i onda to proračunava u sljedećem okviru
- To u biti pomiče sektor promatranja/opažanja u smjeru u kojem se objekt kreće
- Bitni pojmovi: Središte (centar) i centroid

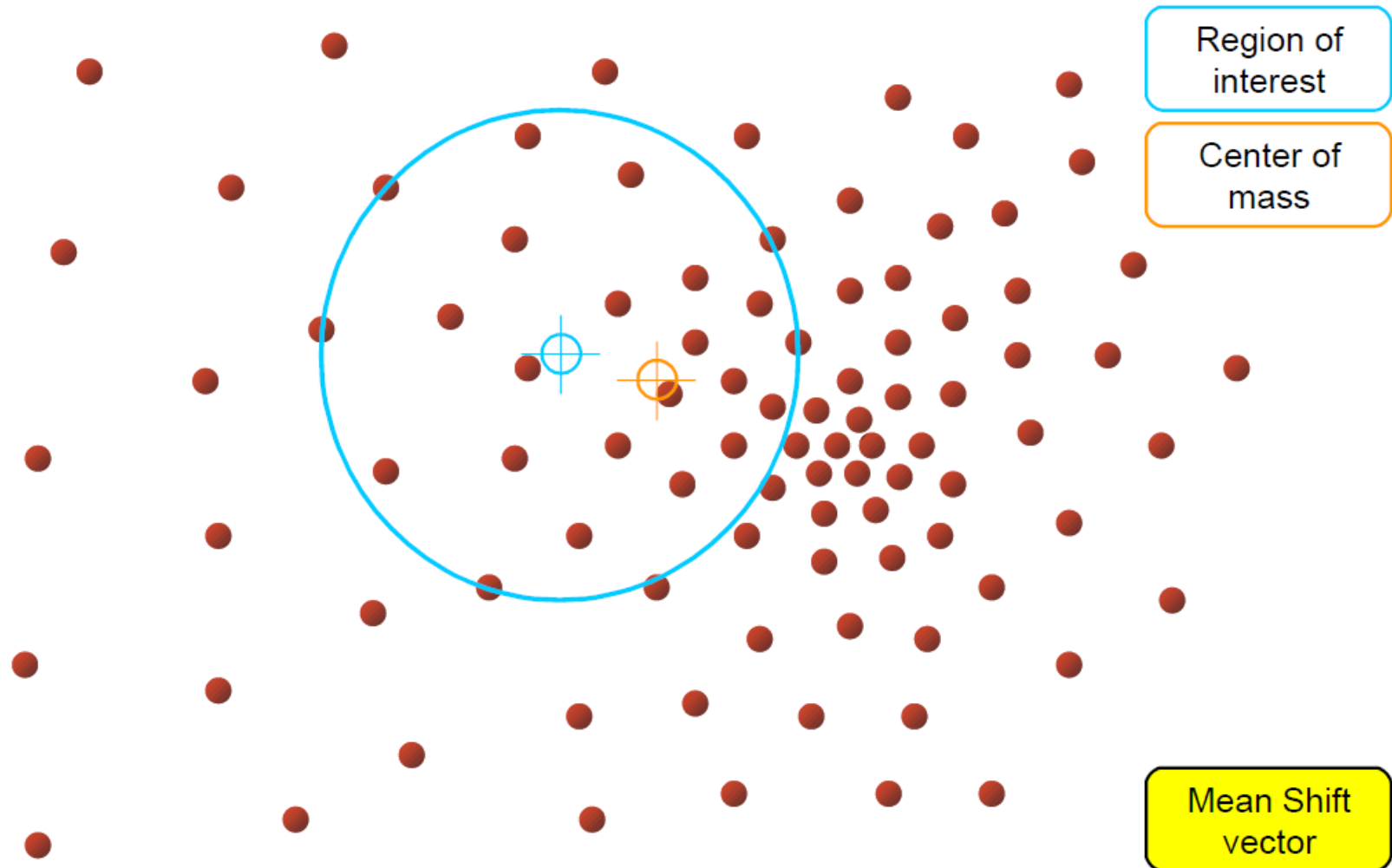


Meanshift algoritam za praćenje pokrenih objekata



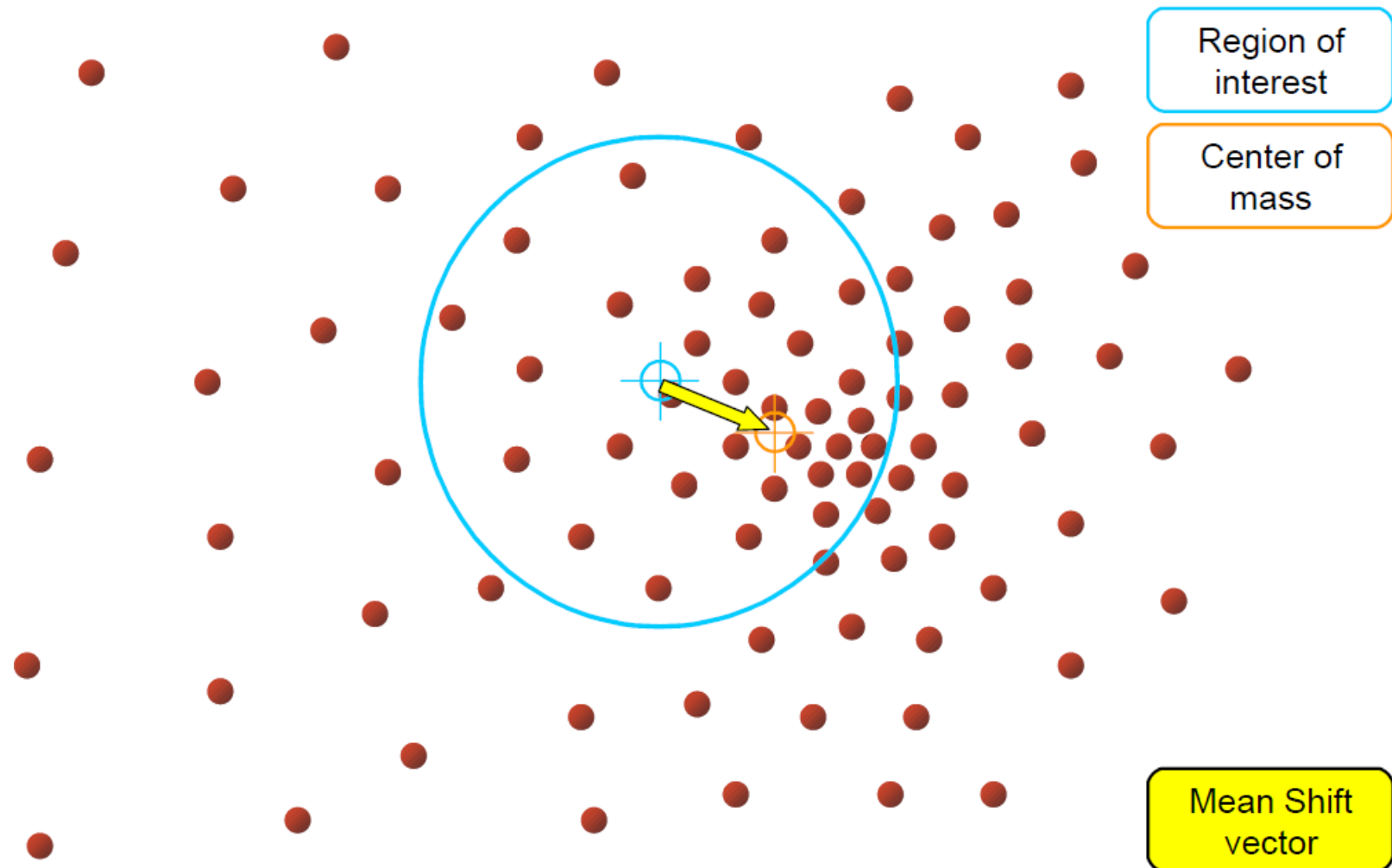
Objective : Find the densest region

Meanshift algoritam za praćenje pokrenih objekata



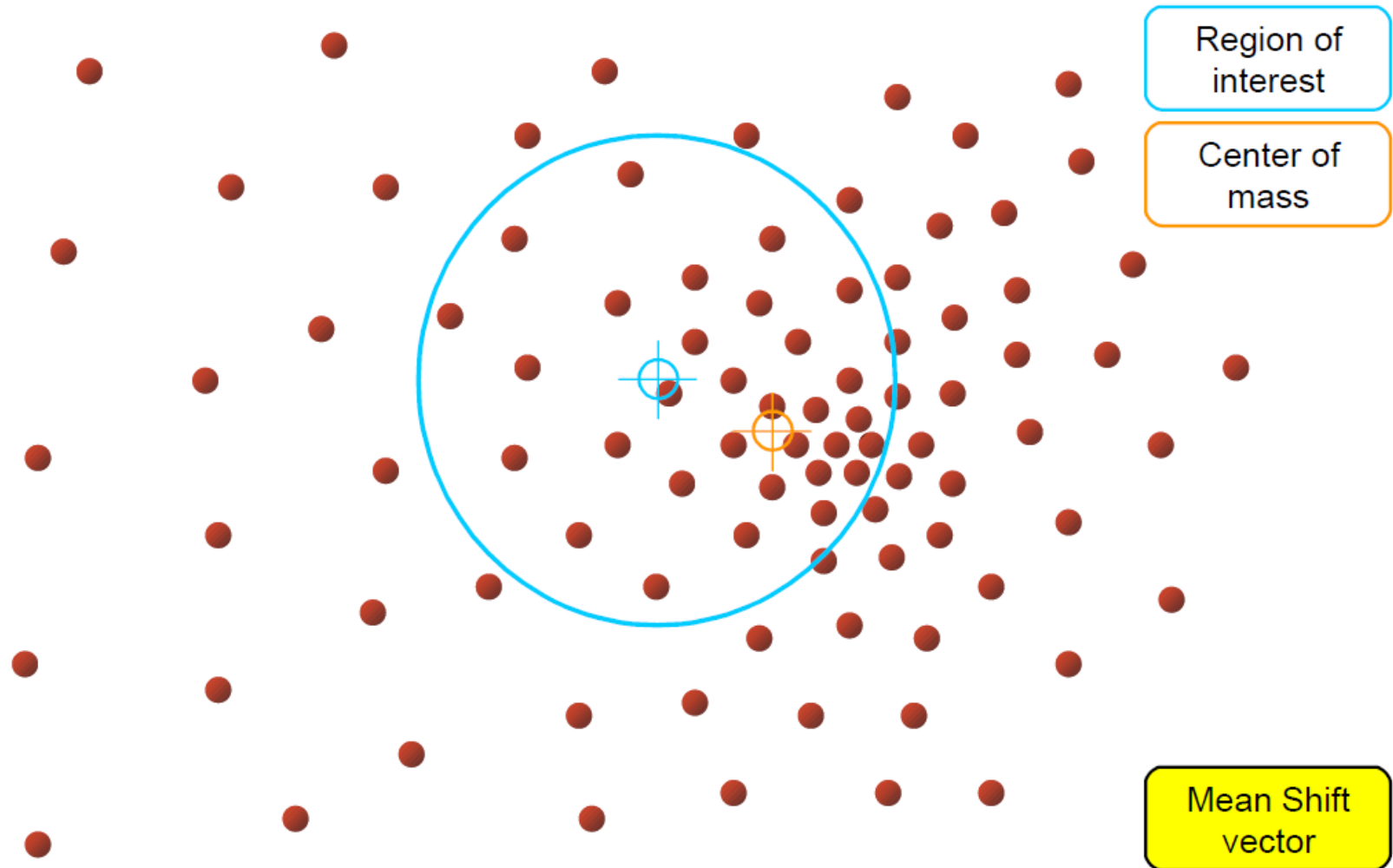
Objective : Find the densest region

Meanshift algoritam za praćenje pokrenih objekata



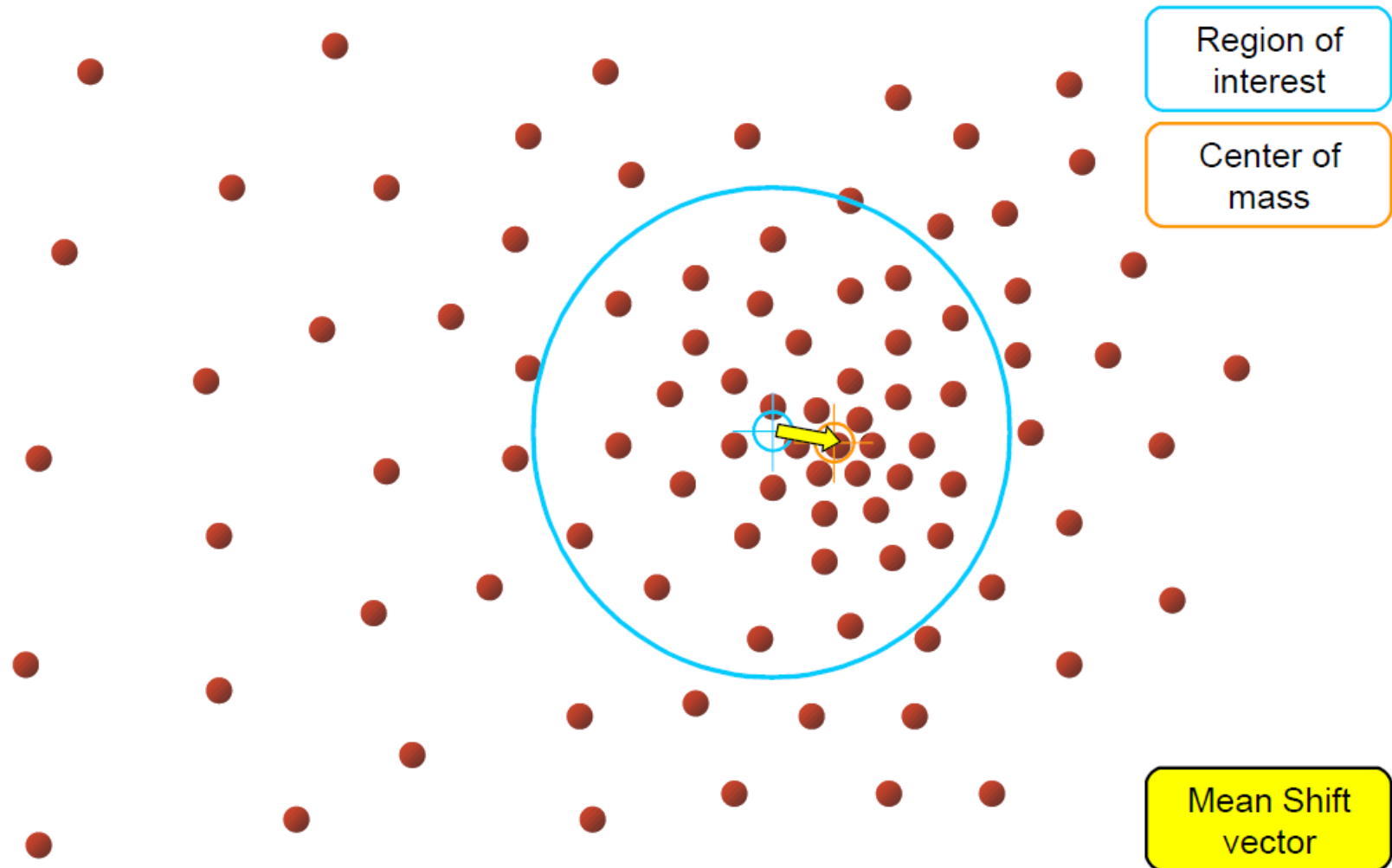
Objective : Find the densest region

Meanshift algoritam za praćenje pokrenih objekata



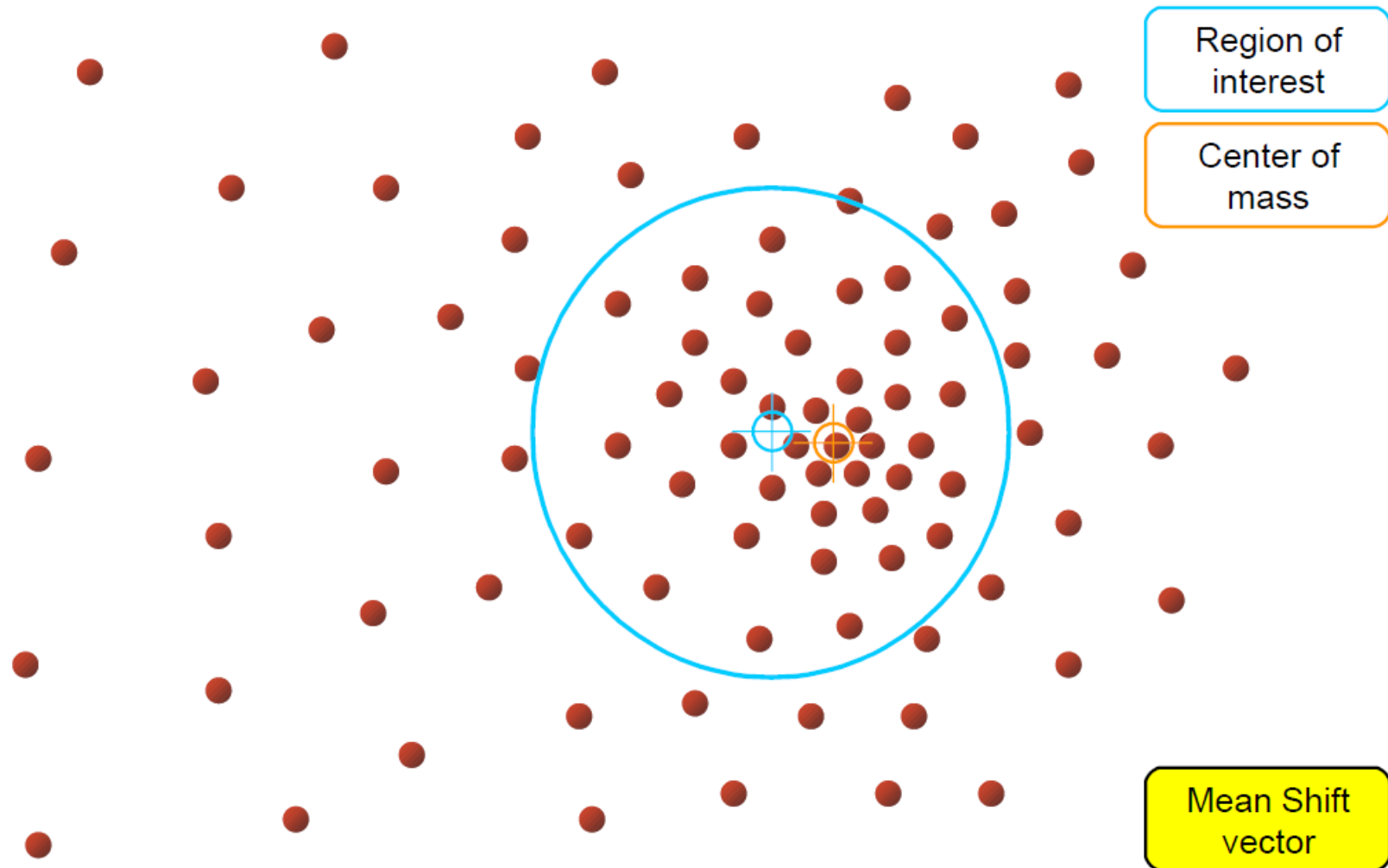
Objective : Find the densest region

Meanshift algoritam za praćenje pokrenih objekata



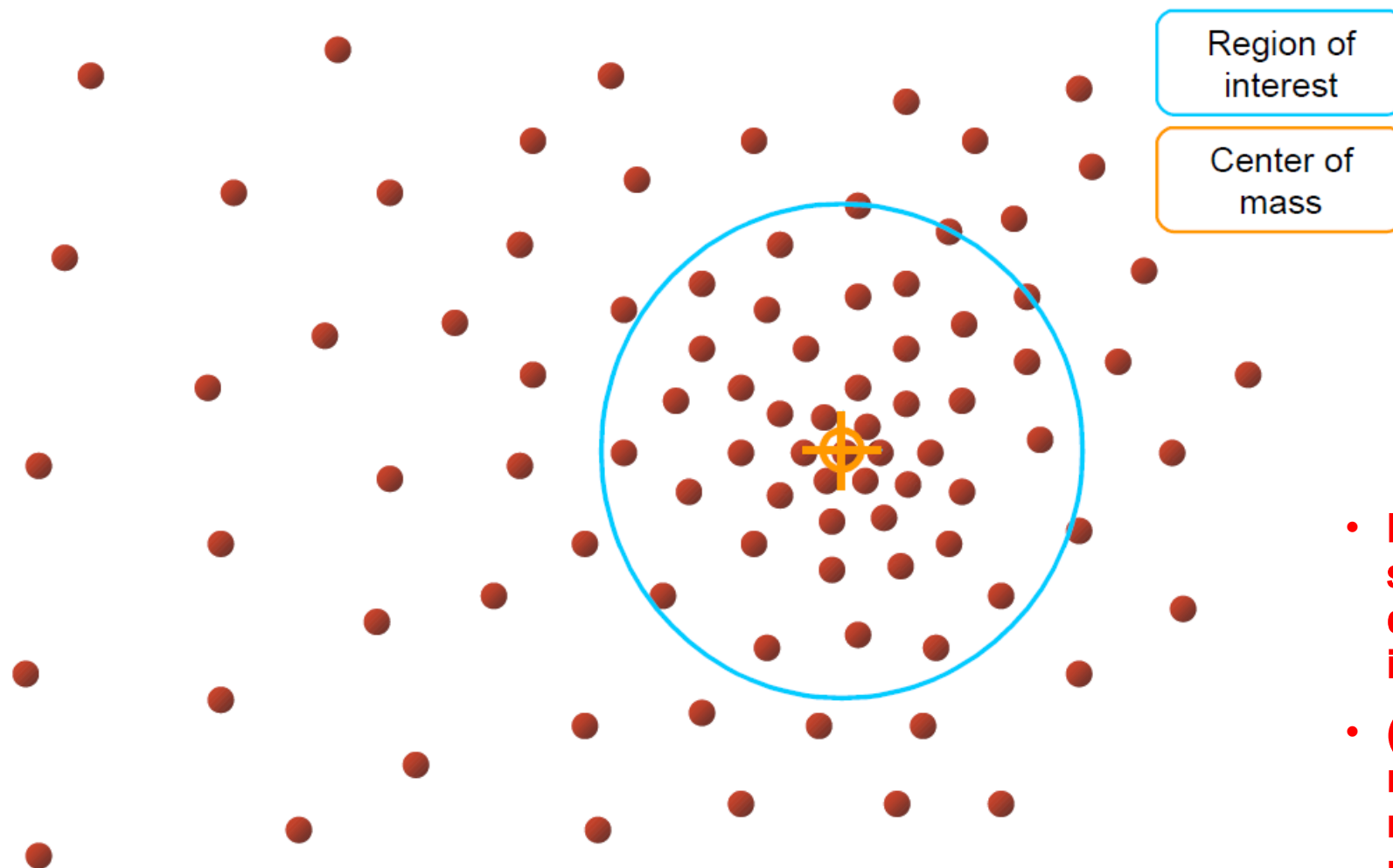
Objective : Find the densest region

Meanshift algoritam za praćenje pokrenih objekata



Objective : Find the densest region

Meanshift algoritam za praćenje pokrenih objekata



Objective : Find the densest region

- Kraj je kada su centar i centroid na istoj lokaciji
- (ili se razlikuju za neku malu predefiniranu vrijednost)

Meanshift algoritam za praćenje pokrenih objekata

- Kraj je kada su centar i centroid na istoj lokaciji (ili se razlikuju za neku malu dozvoljenu vrijednost koju definiramo)
- Što se time dobiva ?
 - prozor s maksimalnom razdiobom piksela koji su prema nekom kriteriju najbližiji traženom objektu, tj. plavi krug na prethodnom slajdu → **ima najveći broj točaka koje vjerojatno pripadaju traženom objektu**

Meanshift algoritam za praćenje pokrenih objekata

Korištenje Meanshift-a na modelima boje

- **Pristup (1)**

- Kreirati sliku „vjerojatnost“, s pikselima kojima su dodijeljene težine prema sličnosti sa željenom bojom (najbolje će raditi za jednoboje objekte)
 - Ne mora biti binarna odluka (1/0) da neki piksel potencijalno pripada/ne pripada objektu
 - Svakom pikselu se može prijedliti broj između 0 i 1 (0% - 100%) koji kaže kolika je vjerojatnost da točka potencijalno pripada objektu kojeg želimo pratiti

- **Pristup (2)**

- Predstaviti raspodjelu boje histogramom → koristiti meanshift da pronade područje koje ima najsličniju raspodjelu boja (najsličniji histogram)

Meanshift algoritam za praćenje pokrenih objekata

Korištenje Meanshift-a na modelima boje – Pristup 1

- Kreirati sliku „vjerojatnost“, s pikselima kojima su dodijeljene težine prema sličnosti sa željenom bojom (najbolje će raditi za jednobojne objekte)
- Idealno - želimo dobiti vrijednost 1 za piksele koji pripadaju objektu kojeg pratimo, a 0 za sve ostale piksele
- Umjesto toga – računamo mapu vjerojatnosti gdje je vrijednost na lokaciji pojedinog piksela proporcionalna vjerojatnosti da taj piksel pripada objektu kojeg pratimo
- Proračun vjerojatnosti može biti zasnovan na
 - Boji
 - Teksturi
 - Obliku (granicama objekta)
 - Predviđenoj lokaciji
 - ... Ili njihovoj kombinaciji



Meanshift algoritam za praćenje pokrenih objekata

Korištenje Meanshift-a na modelima boje – Pristup 1

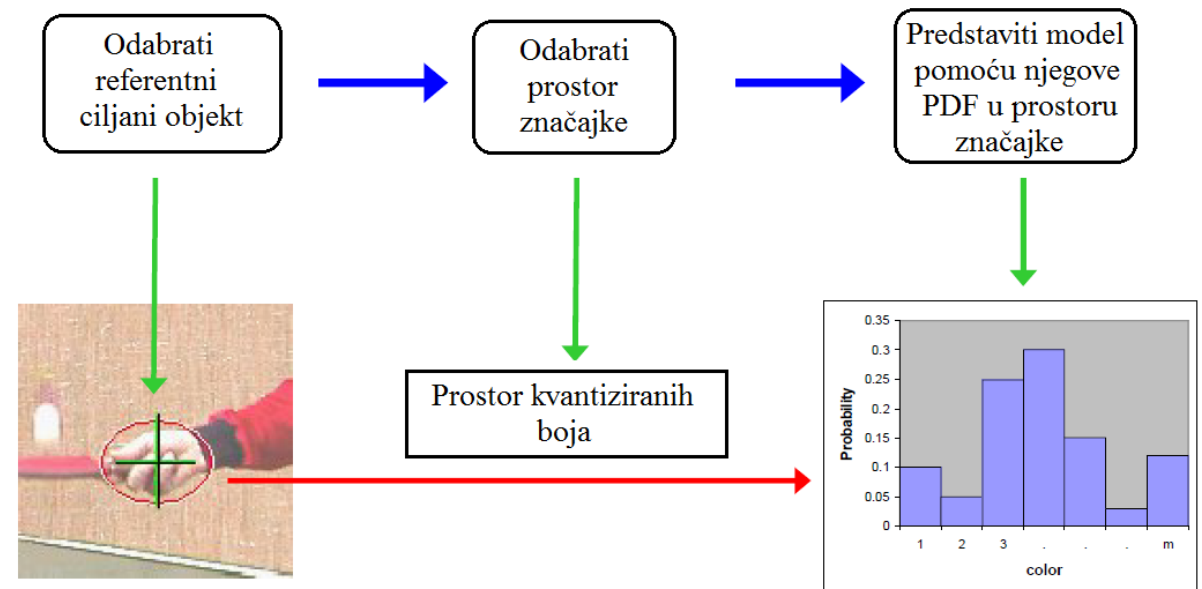
- Meanshift algoritam traži dio okvira određenih (predviđenih) dimenzija u kojem ima najviše piksela koji prema odabranom kriteriju mogu pripadati objektu (uzima u obzir broj piksela i njihove težine (vjerojatnosti))
- **Obratiti pažnju**
 - Objekt koji tražimo - traži se lokacija u mapi „vjerojatnost“, a ne u originalnoj slici



Meanshift algoritam za praćenje pokrenih objekata

Korištenje Meanshift-a na modelima boje – Pristup 2

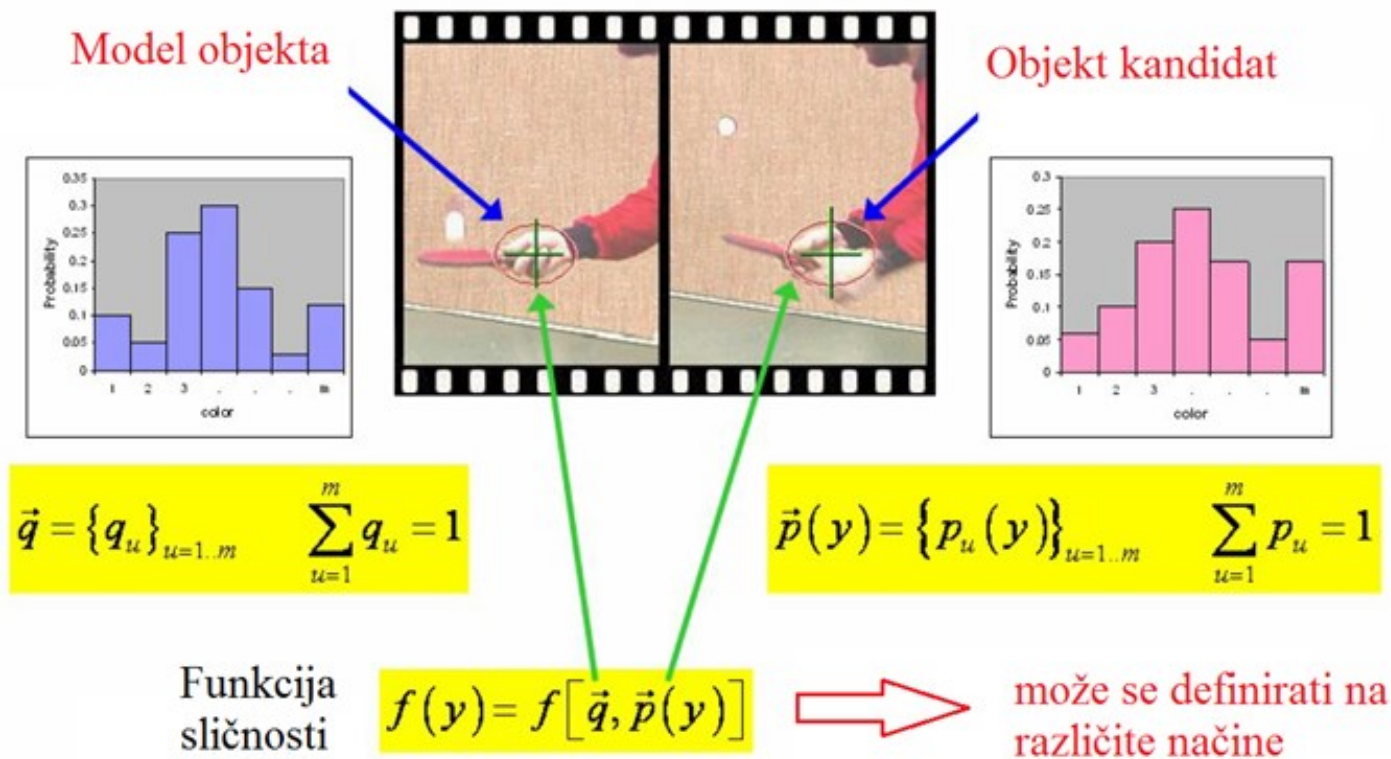
- Predstaviti raspodjelu boje histogramom → koristiti Meanshift da pronade područje koje ima najsličniju raspodjelu boja
- Predstavljanje ciljanog objekta



Meanshift algoritam za praćenje pokrenih objekata

Korišćenje Meanshift-a na modelima boje – Pristup 2

- Predstavljanje PDF (Probability Density Function)



Meanshift algoritam za praćenje pokrenih objekata

Primjer 1 - Meanshift Object Tracking (dostupan od OpenCV 3.0+ verzije pa nadalje)

```
import numpy as np
import cv2

# Initialize webcam
cap = cv2.VideoCapture(0)

# take first frame of the video
ret, frame = cap.read()
print type(frame)

# setup default location of window
r, h, c, w = 240, 100, 400, 160
track_window = (c, r, w, h)

# Crop region of interest for tracking
roi = frame[r:r+h, c:c+w]

# Convert cropped window to HSV color space
hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

# Create a mask between the HSV bounds
lower_purple = np.array([125,0,0])
upper_purple = np.array([175,255,255])
mask = cv2.inRange(hsv_roi, lower_purple, upper_purple)

# Obtain the color histogram of the ROI
roi_hist = cv2.calcHist([hsv_roi], [0], mask, [180], [0,180])

# Normalize values to lie between the range 0, 255
cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)

# Setup the termination criteria
# We stop calculating the centroid shift after ten iterations
# or if the centroid hasn't moved at least 1 pixel
term_crit = ( cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 1, 10 )
```

```
while True:

    # Read webcam frame
    ret, frame = cap.read()

    if ret == True:

        # Convert to HSV
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # Calculate the histogram back projection
        # Each pixel's value is it's probability
        dst = cv2.calcBackProject([hsv],[0],roi_hist,[0,180],1)

        # apply meanshift to get the new location
        ret, track_window = cv2.meanShift(dst, track_window, term_crit)

        # Draw it on image
        x, y, w, h = track_window
        cv2.rectangle(frame, (x,y), (x+w, y+h), 255, 2)

        cv2.imshow('Meansift Tracking', frame)

        if cv2.waitKey(1) == 13: #13 is the Enter Key
            break

    else:
        break

cv2.destroyAllWindows()
cap.release()
```

Meanshift algoritam za praćenje pokrenih objekata

Primjer 1 - Meanshift Object Tracking (dostupan od OpenCV 3.0+ verzije pa nadalje)

```
import numpy as np
import cv2

# Initialize webcam
cap = cv2.VideoCapture(0)

# take first frame of the video
ret, frame = cap.read()
print type(frame)

# setup default location of window
r, h, c, w = 240, 100, 400, 160
track_window = (c, r, w, h)

# Crop region of interest for tracking
roi = frame[r:r+h, c:c+w]

# Convert cropped window to HSV color space
hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

# Create a mask between the HSV bounds
lower_purple = np.array([125,0,0])
upper_purple = np.array([175,255,255])
mask = cv2.inRange(hsv_roi, lower_purple, upper_purple)

# Obtain the color histogram of the ROI
roi_hist = cv2.calcHist([hsv_roi], [0], mask, [180], [0,180])

# Normalize values to lie between the range 0, 255
cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)

# Setup the termination criteria
# We stop calculating the centroid shift after ten iterations
# or if the centroid hasn't moved at least 1 pixel
term_crit = ( cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 1, 10 )
```

- Inicijalizacija web kamere
- Pažnja – mnoštvo inicijalizacija se radi prije same *while* petlje
- Razlog tome - Meanshift mora biti postavljen/definiran ranije
 - Inicijalizacija početne lokacije (tj. prozora gdje očekujemo da se traženi objekt pojavi)
 - Izdvajanje regije interesa za praćenje u ulaznom okviru videa (frame) – samo taj dio okvira
 - Prebacivanje izdvojene regije u HSV
 - Definiramo filter za boju objekta kojeg očekujemo i kreiramo masku (*mask*) – **pogledati prethodno predavanje o filtriranju po boji**
 - *mask* je binarna matrica u kojoj su naznačene lokacije elemenata slike koji imaju boju iz područja propusnosti filtra (sličnu boju objektu kojeg želimo pratiti)

Meanshift algoritam za praćenje pokrenih objekata

Primjer 1 - Meanshift Object Tracking (dostupan od OpenCV 3.0+ verzije pa nadalje)

```
import numpy as np
import cv2

# Initialize webcam
cap = cv2.VideoCapture(0)

# take first frame of the video
ret, frame = cap.read()
print type(frame)

# setup default location of window
r, h, c, w = 240, 100, 400, 160
track_window = (c, r, w, h)

# Crop region of interest for tracking
roi = frame[r:r+h, c:c+w]

# Convert cropped window to HSV color space
hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

# Create a mask between the HSV bounds
lower_purple = np.array([125,0,0])
upper_purple = np.array([175,255,255])
mask = cv2.inRange(hsv_roi, lower_purple, upper_purple)

# Obtain the color histogram of the ROI
roi_hist = cv2.calcHist([hsv_roi], [0], mask, [180], [0,180])

# Normalize values to lie between the range 0, 255
cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)

# Setup the termination criteria
# We stop calculating the centroid shift after ten iterations
# or if the centroid hasn't moved at least 1 pixel
term_crit = ( cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 1, 10 )
```

- Računanje i normalizacija histograma
 - Bitan korak zbog načina na koji Meanshift funkcioniра
 - Računa se histogram samo od izrezanog dijela ulaznog okvira i to samo za one piksele koji su u matrici *mask* imali vrijednost 1
 - „rasteže“ se raspon histograma na cijeli spektar 0-255
- *term_crit* – kriterij za sami Meanshift – kada da stane s praćenjem
 - Meanshift radi više iteracija traženja „najgušćeg“ područja između dva okvira (traženja centroida)
 - Staje nakon 10 iteracija ili nakon što se centroid nije pomaknuo barem za jedan piksel

Meanshift algoritam za praćenje pokrenih objekata

Primjer 1 - Meanshift Object Tracking (dostupan od OpenCV 3.0+ verzije pa nadalje)

- Implementacija same Meanshift funkcije
- Crtanje novog pravokutnika u kojem je objekt
- *calcHist* – funkcija koja računa color histogram za sliku (**pogledati prethodna predavanja**)
- *calcBackProject* – nešto složenije funkcija
 - Prolazi kroz cijeli novi ulazni okvir i na lokaciji svakog piksela daje vjerojatnost da on pripada objektu kojeg tražimo → rezultat je određena vrijednost (često i binarna odluka 0 ili 1) → pohranjeno u *dst*
 - Bijeli pikseli u *dst* predstavljaju zapravo točke na osnovu kojih Meanshift traži mjesta s najvećom gustoćom tih točaka
- *meanShift*
 - Prati objekt na temelju *dst* i na njemu iscrtava novi *track_window*
 - Staje onda kad se ispune uvjeti definirani u *term_crit*

```
while True:

    # Read webcam frame
    ret, frame = cap.read()

    if ret == True:

        # Convert to HSV
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # Calculate the histogram back projection
        # Each pixel's value is it's probability
        dst = cv2.calcBackProject([hsv],[0],roi_hist,[0,180],1)

        # apply meanshift to get the new location
        ret, track_window = cv2.meanShift(dst, track_window, term_crit)

        # Draw it on image
        x, y, w, h = track_window
        cv2.rectangle(frame, (x,y), (x+w, y+h), 255, 2)

        cv2.imshow('Meansift Tracking', frame)

        if cv2.waitKey(1) == 13: #13 is the Enter Key
            break

    else:
        break

cv2.destroyAllWindows()
cap.release()
```


Meanshift algoritam za praćenje pokrenih objekata

Meanshift – vrline i mane

- **Vrline**

- Pogodan za analizu stvarnih podataka
- Ne pretpostavlja nikakav oblik na početku (npr. elipsa, krug,...) na klasteru podataka
- Može raditi s proizvoljnim značajkama (ne samo s bojom)

- **Mane**

- Veličina prozora (odabir područja) nije trivijalan problem
- Stalno pretpostavlja istu veličinu prozora, što može biti problem ako se objekt približava kameri ili se udaljava od nje
- Neprikladna veličina prozora može uzrokovati da potencijalne različite lokacije budu spojene u jednu

P i t a n j a ?

Camshift algoritam

Camnshift algoritam za praćenje pokrenih objekata

- *Continuously Adaptive Meanshift* (Camshift)
- Vrlo sličan Meanshift algoritmu
- Meanshift → jedna od mana mu je fiksna veličina prozora
 - Problematično jer pokret u slikama može biti mali ili veliki
 - Npr. pješak ili automobil se kreću u dubinu dok kamera snima → objekt postaje sve manji i manji → algoritam će imati problema s praćenjem zbog prevelikog prozora
 - Ako je prozor prevelik, možemo promašiti objekt koji želimo pratiti

Camshift algoritam za praćenje pokrenih objekata

- *Camshift* koristi adaptivnu veličinu prozora
 - Prozor mijenja i veličinu i orijentaciju
- Koraci *Camshift* algoritma (pojednostavljeno)
 - Primjenjuje Meanshift dok god on konvergira
 - Računa veličinu prozora
 - Računa orijenataciju prozora
- Naprednija verzija *Camshift* algoritma - ne koristi fiksnu raspodjelu boja nego se prilagođava promjenama koje nastaju na objektu iz okvira u okvir

Camnshift algoritam za praćenje pokrenih objekata

Primjer 2 - Camshift Object Tracking (dostupan od OpenCV 3.0+ verzije pa nadalje)

```
import numpy as np
import cv2

# Initialize webcam
cap = cv2.VideoCapture(0)

# take first frame of the video
ret, frame = cap.read()

# setup default location of window
r, h, c, w = 240, 100, 400, 160
track_window = (c, r, w, h)

# Crop region of interest for tracking
roi = frame[r:r+h, c:c+w]

# Convert cropped window to HSV color space
hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

# Create a mask between the HSV bounds
lower_purple = np.array([130,60,60])
upper_purple = np.array([175,255,255])
mask = cv2.inRange(hsv_roi, lower_purple, upper_purple)

# Obtain the color histogram of the ROI
roi_hist = cv2.calcHist([hsv_roi], [0], mask, [180], [0,180])

# Normalize values to lie between the range 0, 255
cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)

# Setup the termination criteria
# We stop calculating the centroid shift after ten iterations
# or if the centroid has moved at least 1 pixel
term_crit = ( cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 1, 10 )
```

```
while True:

    # Read webcam frame
    ret, frame = cap.read()

    if ret == True:
        # Convert to HSV
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # Calculate the histogram back projection
        # Each pixel's value is it's probability
        dst = cv2.calcBackProject([hsv],[0],roi_hist,[0,180],1)

        # apply Camshift to get the new location
        ret, track_window = cv2.CamShift(dst, track_window, term_crit)

        # Draw it on image
        # We use polylines to represent Adaptive box
        pts = cv2.boxPoints(ret)
        pts = np.int0(pts)
        img2 = cv2.polylines(frame,[pts],True, 255,2)

        cv2.imshow('Camshift Tracking', img2)

        if cv2.waitKey(1) == 13: #13 is the Enter Key
            break

    else:
        break

cv2.destroyAllWindows()
cap.release()
```

Meanshift ili Camnshift ?

- Meanshift
 - Kada imamo konkretne informacije o objektu (veličina objekta s obzirom na poziciju kamere) koji ne mijenja bitno veličinu
- Camshift
 - Kada objekt mijenja svoj oblik i/ili veličinu s obzirom na poziciju kamere i djelomično mijenja svoje karakteristike prema odabranom kriteriju iz okvira u okvir
- Napomena: odabir početne lokacije je često ključan korak
 - Algoritam može zaglaviti na početnoj lokaciji i ne pomaknuti se iz nje
 - Ovaj je problem manje izražen i manje vjerojatan ako znamo lokaciju objekta u prethodnom okviru (**ako smo prethodno obavili detekciju i znamo lokaciju**)

P i t a n j a ?

***Optical flow* algoritam**

Algoritmi za praćenje pokrenih objekata u sceni

Podsjetimo se...

- Scene s pokretnim objektima
 - Analiza scene puno složenija nego za stacionarne scene (varijacije intenziteta moraju se uzeti u obzir)
 - Segmentirati pokretne objekte na temelju njihova pokreta – ne čini se kao težak zadatak !
 - Razlika susjednih okvira videa bi to trebala omogućiti – međutim, nije tako jednostavno

Algoritmi za praćenje pokrenih objekata u sceni

Podsjetimo se...

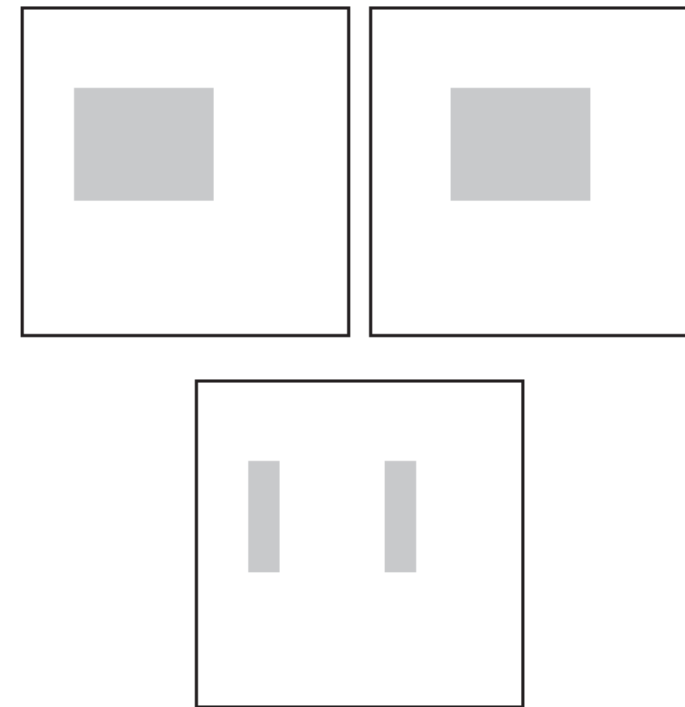
Primjer – objekt se pomiče s lijeva na desno (gornje dvije slike)

- Razlika ta dva okvira (donja slika):

(I) područja konstantnog intenziteta ne odaju znakove pokreta

(II) rubovi paralelni sa smjerom pokreta ne odaju znakove pokreta

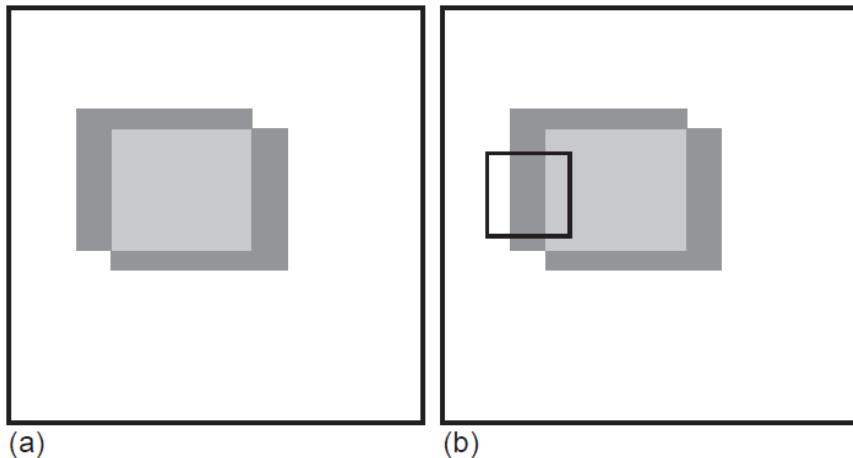
- Samo rubovi koji imaju komponentu okomitu na smjer gibanja daju informacije o pokretu)



Algoritmi za praćenje pokrenih objekata u sceni

Podsjetimo se ... (problem otvora – *aperture problem*)

- Nadalje – postoji neodređenost oko smjera samog vektora brzine
- Ovo se javlja djelomično zbog toga što je premalo informacija dostupno u nekom malom otvoru, kako bi se omogućilo izračunavanje punog vektora brzine → problem otvora (*aperture problem*)



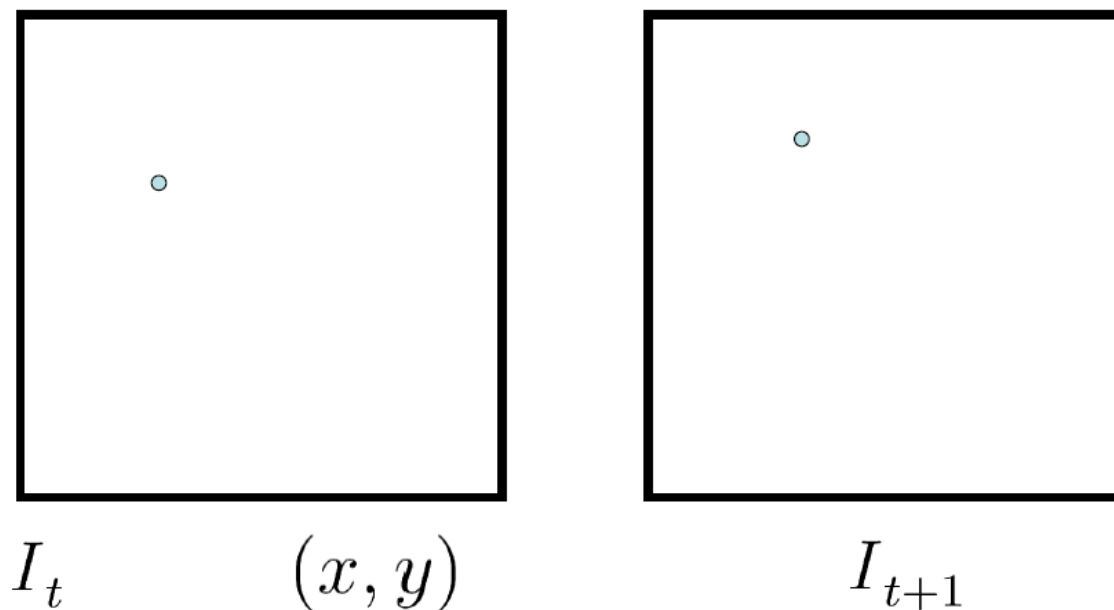
- Slika (a)
(Tamno siva) područja pokreta objekta čiji središnji uniformni dio (svijetlo sivi) ne odaje znakove pokreta
- Slika (b)
Prikaz kako se malo toga o smjeru pokreta može zaključiti iz malog prozora (crni okvir) → dovodi do neodređenosti o zaključivanju o smjeru samog pokreta objekta

Optical flow algoritam za praćenje pokrenih objekata

- Pokušava riješiti probleme navedene na prethodna dva slajda
- Optical flow – ideja:
 - Primijeniti nekakav lokalni operator na sve piksele, što dovodi do dobivanja 2D polja vektora pokreta koji lagano (glatko) variraju duž cijele slike
 - 2D polje vektora koji zapravo opisuju translaciju unutar slike
- Atrakcija leži u korištenju lokalnog operatora – ne troši mnoštvo računalnih resursa
 - Usporedbe radi - usporedivo s korištenjem nekakvog detektora rubova u nekoj normalnoj slici
 - Naravno, lokalni operator se ovdje primjenjuje na par uzastopnih okvira u video sekvenci
- Broj proračunatih vektora toka (*flow vectors*) jednak je broju piksela u samoj slici (okviru videa)

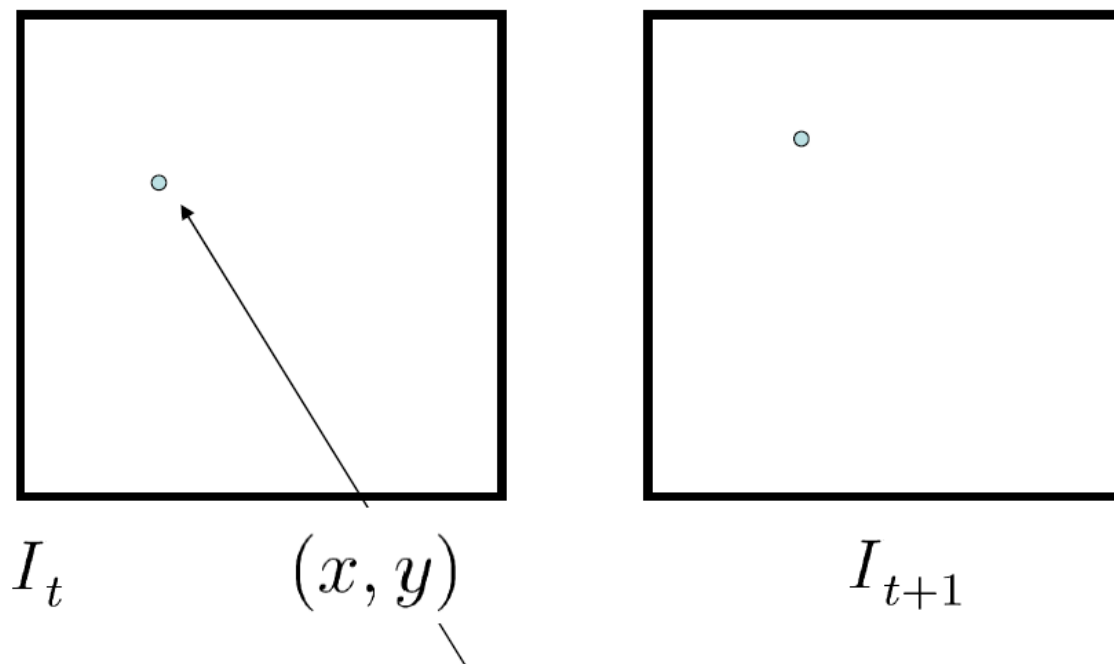
Optical flow algoritam za praćenje pokrenih objekata

- Osnovna postavka
 - 2 uzastopna video okvira



Optical flow algoritam za praćenje pokrenih objekata

- Osnovno pitanje...

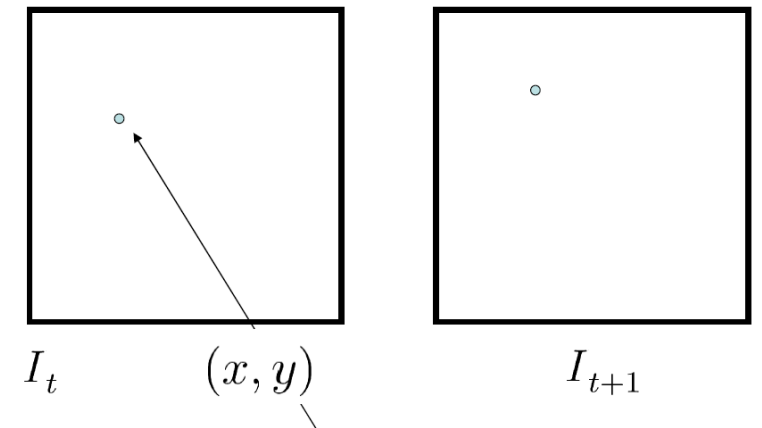


Gdje se pomaknula točka s ove lokacije u sljedećem okviru?

Optical flow algoritam za praćenje pokrenih objekata

Optical flow je zasnovan na 3 pretpostavke

1. Konstantnost intenziteta – intenzitet piksela nekog objekta ne mijenja se između uzastopnih okvira
2. Vrijeme između dva okvira je dovoljno kratko da se pokret može razmatrati koristeći razliku okvira
3. Prostorna konzistentnost – susjedni elementi slike imaju sličan pokret



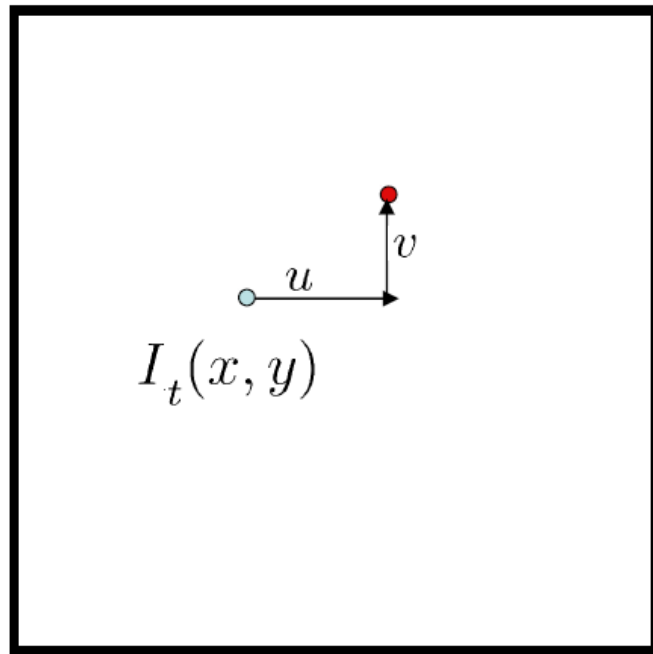
Gdje se pomaknula točka s ove lokacije u sljedećem okviru?

Optical flow algoritam za praćenje pokrenih objekata

- U mnogim slučajevima te pretpostavke „padaju u vodu“, ali za male pokrete i kratko vrijeme između slika (okvira), OF je vrlo često dobar model
- Konstantnost intenziteta – piksel koji pripada nekom objektu $I(x,y,t)$ u trenutku t ima isti intenzitet u trenutku $t+\Delta t$, nakon pokreta/pomaka za $(\Delta x, \Delta y)$
- Jednadžba konstantnosti intenziteta $I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$

Optical flow algoritam za praćenje pokrenih objekata

- Optical flow → odrediti u i v za svaki piksel

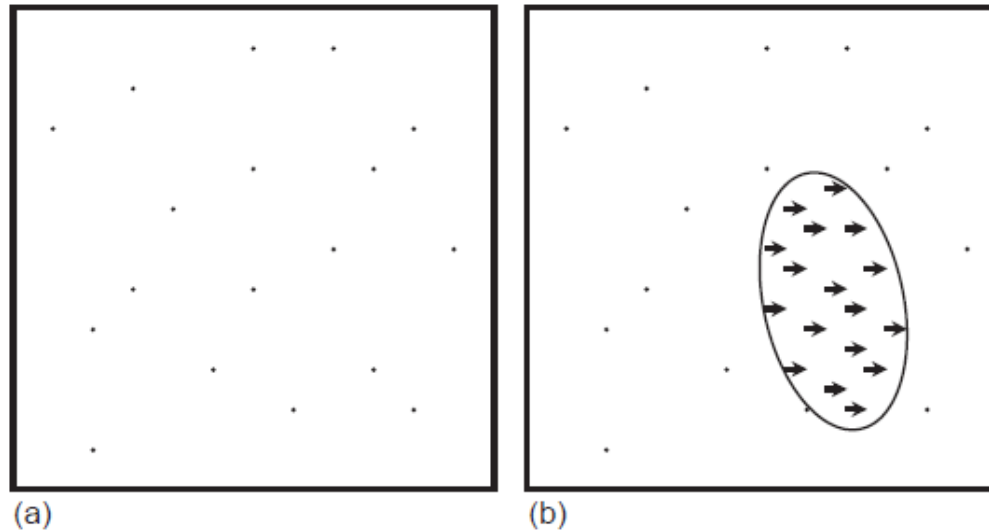


- Uzimajući u obzir prethodne pretpostavke...
- I uz određene matematičke operacije...
- Potrebno je odrediti u i v za svaki piksel
 - Postupak određivanja u i v → **nećemo ovdje ići u detalje → dostupno u literaturi**
- **Pretpostavka** – susjedni pikseli izloženi su sličnom pokretu (ako pripadaju istom objektu) – na taj način pokušavaju se pratiti objekti

Optical flow algoritam za praćenje pokrenih objekata

Optical flow

- Interpretacija polja optičkog toka (duljina strelice ukazuje na iznos/brzinu vektora pokreta)

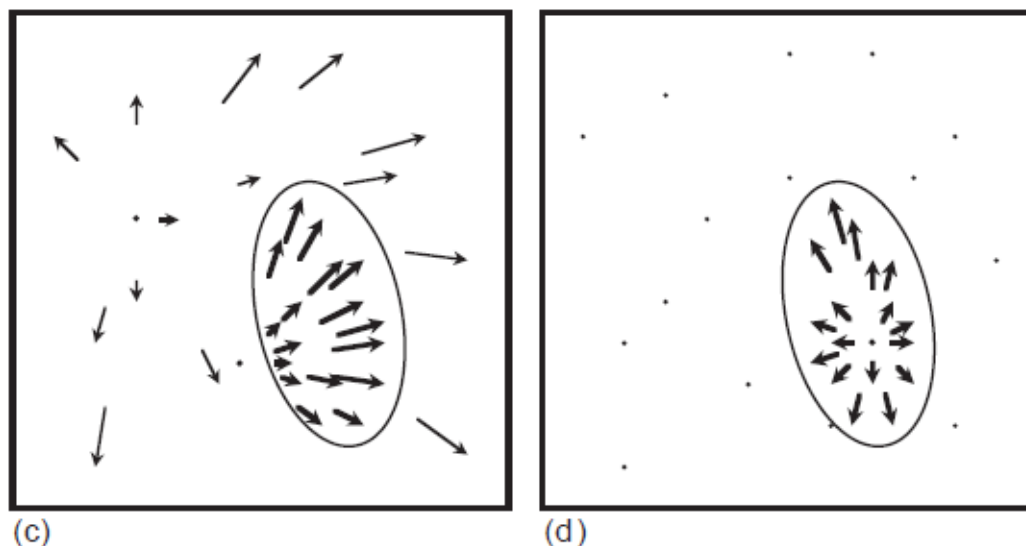


- (a) objekt se ne kreće (u i v su nula za gotovo sve piksele scene/svih objekata)
- (b) jedan objekt se kreće u desno

Optical flow algoritam za praćenje pokrenih objekata

Optical flow

- Interpretacija polja optičkog toka (duljina strelice ukazuje na iznos/brzinu vektora pokreta)

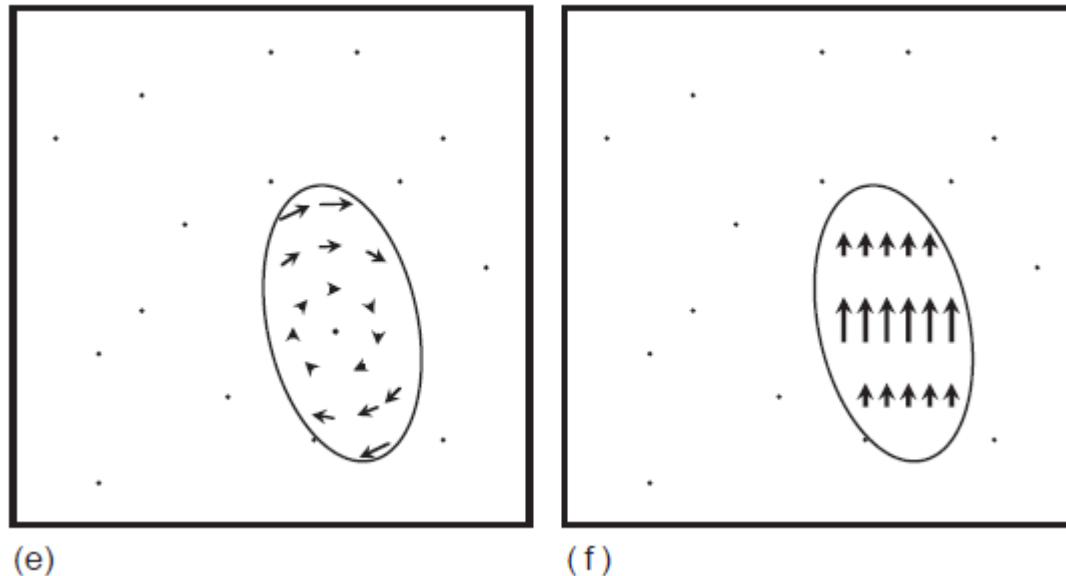


- (c) Kamera se kreće prema sceni – značajke stacionarnog objekta počinju divergirati od fokusa, dok se jedan veliki objekt kreće ispred kamere i udaljava od fokusa
- (d) jedan objekt se kreće izravno prema stacionarnoj kameri

Optical flow algoritam za praćenje pokrenih objekata

Optical flow

- Interpretacija polja optičkog toka (duljina strelice ukazuje na iznos/brzinu vektora pokreta)

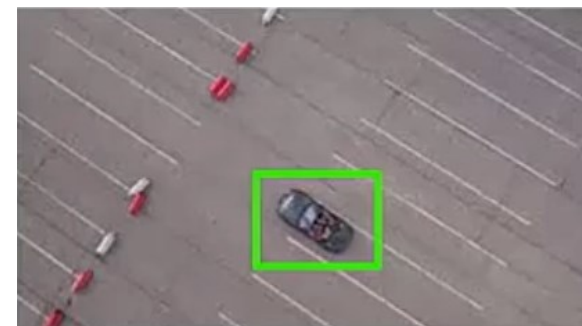


- (e) objekt rotira oko *Line of sight* osi kamere
- (f) objekt rotira oko osi okomite na *Line of sight* os kamere

Optical flow algoritam za praćenje pokrenih objekata

Optical flow (OF) & OpenCV

- OpenCV implementira 2 različita OF algoritma
- ***Lucas-Kanade Differential Method***
 - Prati neke ključne točke (KT) u videu (prethodi mu detekcija KT)
 - Dobar za *corner-like* značajke (npr praćenje auta dronovima – top-down perspektiva)
- ***Dense Optical Flow***
 - Sporiji, ali računa OF za sve točke okvira, za razliku od Lucas-Kanade koji uglavnom prati samo određene značajke (npr. uglove)
 - Boje (Hue) se koriste za iskaz smjera pokreta, a svjetlina/intenzitet (Value) za iskaz brzine

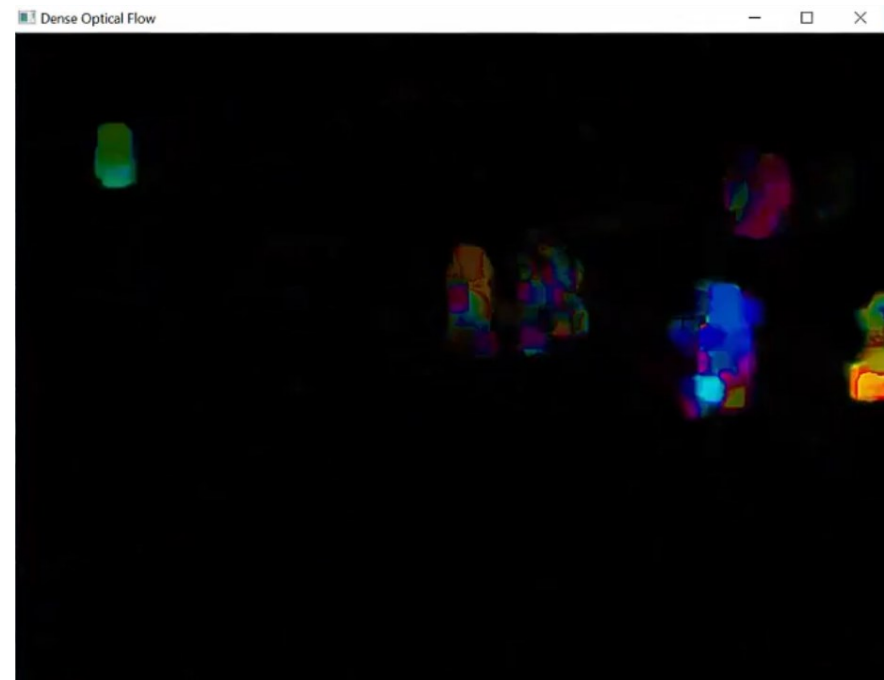
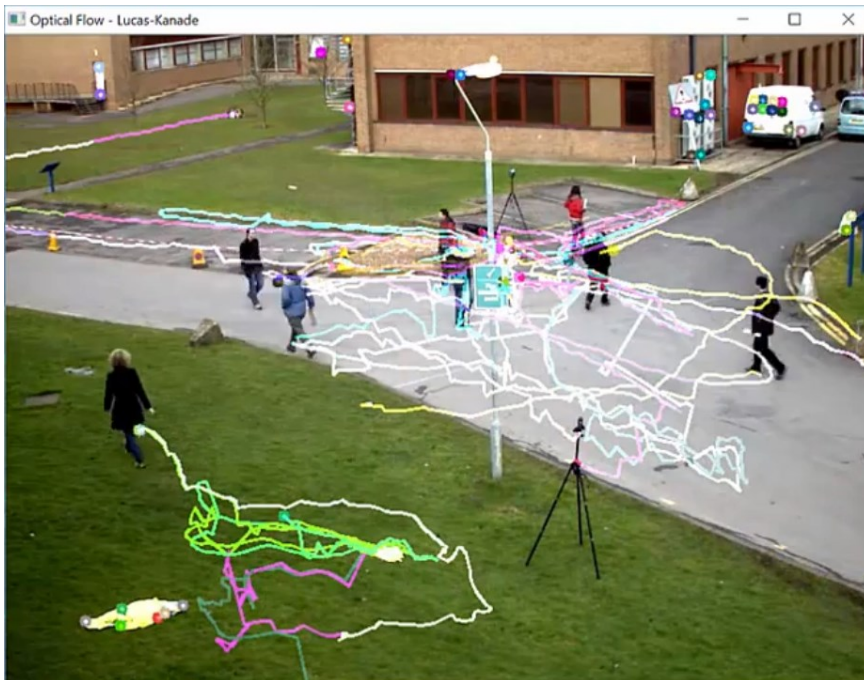


Optical flow algoritam za praćenje pokrenih objekata

Optical flow (OF) & OpenCV

Lucas-Kanade Differential Method &

Dense Optical Flow



Optical flow algoritam za praćenje pokrenih objekata

Primjer 3 - Lucas-Kanade Optical Flow u OpenCV (OpenCV 3.1 verzija pa nadalje)

```
import numpy as np
import cv2

# Load video stream
cap = cv2.VideoCapture('images/walking.avi')

# Set parameters for ShiTomasi corner detection
feature_params = dict( maxCorners = 100,
                       qualityLevel = 0.3,
                       minDistance = 7,
                       blockSize = 7 )

# Set parameters for Lucas kanade optical flow
lucas_kanade_params = dict( winSize = (15,15),
                             maxLevel = 2,
                             criteria = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03))

# Create some random colors
# Used to create our trails for object movement in the image
color = np.random.randint(0,255,(100,3))

# Take first frame and find corners in it
ret, prev_frame = cap.read()
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

# Find initial corner locations
prev_corners = cv2.goodFeaturesToTrack(prev_gray, mask = None, **feature_params)

# Create a mask image for drawing purposes
mask = np.zeros_like(prev_frame)
```

```
while(1):
    ret, frame = cap.read()
    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # calculate optical flow
    new_corners, status, errors = cv2.calcOpticalFlowPyrLK(prev_gray,
                                                            frame_gray,
                                                            prev_corners,
                                                            None,
                                                            **lucas_kanade_params)

    # Select and store good points
    good_new = new_corners[status==1]
    good_old = prev_corners[status==1]

    # Draw the tracks
    for i,(new,old) in enumerate(zip(good_new, good_old)):
        a, b = new.ravel()
        c, d = old.ravel()
        mask = cv2.line(mask, (a,b),(c,d), color[i].tolist(), 2)
        frame = cv2.circle(frame, (a,b), 5, color[i].tolist(),-1)

    img = cv2.add(frame,mask)

    # Show Optical Flow
    cv2.imshow('Optical Flow - Lucas-Kanade',img)
    if cv2.waitKey(1) == 13: #13 is the Enter Key
        break

    # Now update the previous frame and previous points
    prev_gray = frame_gray.copy()
    prev_corners = good_new.reshape(-1,1,2)

cv2.destroyAllWindows()
cap.release()
```


Optical flow algoritam za praćenje pokrenih objekata

Primjer 3 - *Lucas-Kanade Optical Flow* u OpenCV (OpenCV 3.1 verzija pa nadalje)

- Definiranje parametara za *ShiTomasi* detekciju uglova (koji se koristi u Lucas-Kanade algoritmu)
- Definiranje parametara za Lucas-Kanade OF
- Definiranje boja kojima će se iscrtavati putanja kretanja različitih praćenih objekata
- Pronalaženje uglova u prvom okviru
- Petlja
 - Tražimo uglove u sljedećem okviru (koristimo prethodno definirane parametre)
 - Pohranjujemo stare i nove lokacije uglova
 - Iscrtavamo kretanje koristeći lokacije uglova

Optical flow algoritam za praćenje pokrenih objekata

Primjer 4 - *Dense Optical Flow* u OpenCV (OpenCV 3.1 verzija pa nadalje)

- Kraći kod – ne treba tražiti uglove jer računa OF za svaki piksel

```
import cv2
import numpy as np

# Load video stream
cap = cv2.VideoCapture("images/walking.avi")

# Get first frame
ret, first_frame = cap.read()
previous_gray = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)
hsv = np.zeros_like(first_frame)
hsv[...,1] = 255
```

```
while True:

    # Read of video file
    ret, frame2 = cap.read()
    next = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    # Computes the dense optical flow using the Gunnar Farneback's algorithm
    flow = cv2.calcOpticalFlowFarneback(previous_gray, next,
                                         None, 0.5, 3, 15, 3, 5, 1.2, 0)

    # use flow to calculate the magnitude (speed) and angle of motion
    # use these values to calculate the color to reflect speed and angle
    magnitude, angle = cv2.cartToPolar(flow[...,0], flow[...,1])
    hsv[...,0] = angle * (180 / (np.pi/2))
    hsv[...,2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
    final = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    # Show our demo of Dense Optical Flow
    cv2.imshow('Dense Optical Flow', final)
    if cv2.waitKey(1) == 13: #13 is the Enter Key
        break

    # Store current image as previous image
    previous_gray = next

cap.release()
cv2.destroyAllWindows()
```

P i t a n j a ?